

paint.py

```
1  from __future__ import annotations
2  from typing import TYPE_CHECKING
3  from network import NetworkHandler
4  from dataclasses import asdict
5  from tkinter import filedialog
6  from PIL import Image, ImageTk
7  from colorbar import ColorBar
8  from toolbar import ToolBar
9  from point import Point
10 import tkinter as tk
11 import numpy as np
12 import actions
13
14 if TYPE_CHECKING:
15     from tools import Tool
16     from client import App
17
18 class Paint(tk.Frame):
19     def __init__(self, master: App, wsUrl: str, network = False, *args, **kwargs):
20         super().__init__(master = master, *args, **kwargs)
21
22         self.configure(background = "gray")
23
24         self.master = master
25
26         self.width = 600
27         self.height = 600
28
29         self.pixel_size = 4
30
31         self.undoStack = []
32         self.redoStack = []
33
34         self.lastPos: Point = Point(0, 0)
35         self.currentStroke = []
36
37         # Toolbar + Colorbar
38
39         self.colorBar = ColorBar(self, self.setColor)
40         self.colorBar.pack(side = "bottom", fill = "x")
41
42         self.toolBar = ToolBar(self, self.setTool)
43         self.toolBar.pack(side = "left", fill = "y")
44
45         self.currentTool = self.toolBar.getTool()
46         self.currentColor: tuple[int, int, int] = self.colorBar.getColor()
47
```

```

48         # Canvas Buffer and Image
49
50         self.canvas = tk.Canvas(self,
51             width = self.width,
52             height = self.height,
53             borderwidth = 0,
54             highlightthickness = 0
55         )
56
57         self.canvas.pack(expand = True, anchor = "nw")
58
59         self.buffer = np.full((int(self.height / self.pixel_size), int(self.width /
self.pixel_size), 3), 255, dtype = np.uint8)
60         self.tempBuffer = np.zeros((int(self.height / self.pixel_size),
int(self.width / self.pixel_size), 3), dtype = np.uint8)
61         self.tempMask = np.zeros((int(self.height / self.pixel_size),
int(self.width / self.pixel_size)), dtype = bool)
62
63         self.image = self.canvas.create_image(0, 0, anchor = "nw")
64
65         self.render()
66
67         # Canvas Events
68
69         self.canvas.bind("<B1-Motion>", self.mouseDrag)
70         self.canvas.bind("<ButtonRelease-1>", self.mouseUp)
71         self.canvas.bind("<ButtonPress-1>", self.mouseDown)
72         self.canvas.bind("<Motion>", self.mouseMove)
73         self.master.bind("<Return>", lambda _: self.keyEnter())
74
75         # Network
76         self.networkHandler = NetworkHandler(self, wsUrl)
77
78         if network:
79             self.networkHandler.run()
80
81         self.master.protocol("WM_DELETE_WINDOW", self.close)
82
83         def save(self):
84             filePath = filedialog.asksaveasfilename(
85                 filetypes = [('PNG Files', '*.png'), ('JPEG Files', '*.jpg')],
86                 defaultextension = ".png",
87                 title = "Save Image As"
88             )
89
90             image = Image.fromarray(self.buffer)
91             image.save(filePath)
92
93         def close(self):

```

```

94         self.master.destroy()
95         self.networkHandler.stop()
96
97     def setTool(self, tool: Tool):
98         self.currentTool = tool
99
100    def setColor(self, color):
101        self.currentColor = color
102
103    def mouseDrag(self, event):
104        event.x = int(event.x / self.pixel_size)
105        event.y = int(event.y / self.pixel_size)
106        self.currentTool.on_mouse_drag(self, event)
107        self.render()
108
109    def mouseUp(self, event):
110        event.x = int(event.x / self.pixel_size)
111        event.y = int(event.y / self.pixel_size)
112        self.currentTool.on_mouse_up(self, event)
113        self.clearTempBuffer()
114        self.render()
115
116    def mouseDown(self, event):
117        event.x = int(event.x / self.pixel_size)
118        event.y = int(event.y / self.pixel_size)
119        self.currentTool.on_mouse_down(self, event)
120        self.lastPos = Point(event.x, event.y)
121        self.render()
122
123    def mouseMove(self, event):
124        event.x = int(event.x / self.pixel_size)
125        event.y = int(event.y / self.pixel_size)
126        self.currentTool.on_mouse_move(self, event)
127
128    def keyEnter(self):
129        self.currentTool.on_enter(self)
130
131    def clearTempBuffer(self):
132        self.tempBuffer.fill(0)
133        self.tempMask.fill(False)
134
135    def rect(self, start: Point, stop: Point, fill = False, temp = False):
136        if temp:
137            self.clearTempBuffer()
138            actions.rect(self.tempBuffer, start, stop, self.currentColor, fill,
self.tempMask)
139
140        else:
141            actions.rect(self.buffer, start, stop, self.currentColor)

```

```

142
143         self.networkHandler.sendAction({
144             "rect": {
145                 "start": asdict(start),
146                 "stop": asdict(stop),
147                 "color": self.currentColor,
148                 "fill": fill
149             }})
150
151
152     def line(self, start: Point, stop: Point, temp = False):
153         if temp:
154             self.clearTempBuffer()
155             actions.line(self.tempBuffer, start, stop, self.currentColor,
self.tempMask)
156
157         else:
158             actions.line(self.buffer, start, stop, self.currentColor)
159
160             self.networkHandler.sendAction({
161                 "line": {
162                     "start": asdict(start),
163                     "stop": asdict(stop),
164                     "color": self.currentColor
165                 }
166             })
167
168     def ellipse(self, start: Point, stop: Point, fill = False, temp = False):
169         if temp:
170             self.clearTempBuffer()
171             actions.ellipse(self.tempBuffer, start, stop, self.currentColor, fill,
self.tempMask)
172
173         else:
174             actions.ellipse(self.buffer, start, stop, self.currentColor, fill)
175
176             self.networkHandler.sendAction({
177                 "ellipse": {
178                     "start": asdict(start),
179                     "stop": asdict(stop),
180                     "color": self.currentColor,
181                     "fill": fill
182                 }
183             })
184
185     def fill(self, point: Point):
186         actions.floodFill(self.buffer, point, self.currentColor)
187
188         self.networkHandler.sendAction({

```

```

189         "fill": {
190             "point": asdict(point),
191             "color": self.currentColor
192         }
193     })
194
195     def onCanvas(self, x, y) -> bool:
196         if 0 <= x * self.pixel_size < self.width and 0 <= y * self.pixel_size <
self.height:
197             return True
198         else:
199             return False
200
201     def render(self):
202         # NumPy -> PIL -> PhotoImage
203
204         # Expand mask into third dimension to make np.where work [x, y, None] =
True | False
205         mask = self.tempMask[..., None]
206
207         pilImage = Image.fromarray(
208             np.where(mask, self.tempBuffer, self.buffer)
209         ).resize(
210             (self.width, self.height),
211             Image.NEAREST # type: ignore
212         )
213
214         self.imageTK = ImageTk.PhotoImage(pilImage)
215
216         self.canvas.itemconfig(self.image, image = self.imageTK)
217
218
219     def runAction(self, data):
220         if "rect" in data:
221             rect = data["rect"]
222             actions.rect(self.buffer, Point(**rect["start"]),
Point(**rect["stop"]), rect["color"], rect["fill"])
223
224         elif "line" in data:
225             line = data["line"]
226             actions.line(self.buffer, Point(**line["start"]),
Point(**line["stop"]), line["color"])
227
228         elif "ellipse" in data:
229             ellipse = data["ellipse"]
230             actions.ellipse(self.buffer, Point(**ellipse["start"]),
Point(**ellipse["stop"]), ellipse["color"], ellipse["fill"])
231
232         elif "fill" in data:
233             fill = data["fill"]

```

```
234         actions.floodFill(self.buffer, Point(**fill["point"]), fill["color"])
235
236     self.render()
```